

Patient Monitoring System (PMS)

Part 2: Architecture extension

1 Part 2 – Extension of the initial architecture

In this part of the project, you will extend the initial PMS-architecture that is provided as part of the assignment. The goal is **not** to merely to add functionality, but to perform a **quality-driven architectural extension** and to explicitly reason about **architectural trade-offs with the original design and within your suggested extensions**. The extension is driven by a number of quality attribute scenarios (QASs). Part 2 consists of the following tasks:

- Select **two quality attribute scenarios (QASs)** of the provided list of QASs (see Section 3): one High priority and one Medium priority.
- Design and document an architectural extension that fully supports your selected QASs as well as the QASs in the initial architecture.
- Critically analyze how your extension **interacts with, modifies, or conflicts with** architectural decisions made in the initial architecture and made as part of your extension.
- Motivate your design decisions using architectural rationale and quality attribute reasoning.

While you are **allowed** to make any necessary change to the original system, you need to take care to not undermine previously-made decisions. **If you change original decisions**, you need to be very explicit about which decisions you changed, the impact of the change, and why it was necessary to introduce that change. Be aware of the priorities of the QASs.

Architectural trade-offs and sensitivity points The reasoning expected in this assignment is inspired by the *Architecture Tradeoff Analysis Method (ATAM)*. ATAM is a well-established architectural evaluation method that focuses on understanding how architectural decisions influence quality attributes.

In particular, ATAM introduces the notions of:

- **Sensitivity points:** architectural decisions or parameters for which small changes can have a large impact on one or more quality attributes (e.g. changing deployment locality may strongly affect performance or availability).
- **Trade-off points:** architectural decisions that positively influence one quality attribute while negatively impacting another, this is often a sensitivity point for more than one attribute (e.g. improving modifiability at the cost of performance or predictability).

You are *not* required to perform a full ATAM analysis. However, you are expected to: (i) identify the sensitivity points introduced or affected by your extension, (ii) explicitly discuss trade-offs between quality attributes, and (iii) justify why these trade-offs are acceptable in the context of the system and the involved QASs.

1.1 What “Fully Supports a Requirement” Means

For the chosen QAS, you must address **all elements of its quality attribute scenario**. Your architectural extension is considered complete *only if the entire scenario is covered end-to-end*, not just the main functionality or a single response.

- **Stimulus:** Clearly identify the event or condition that triggers the scenario (e.g., component failure, overload, unauthorized access attempt, privacy request, peak usage). The stimulus must be realistic and consistent with the PMS context.
- **Artifact:** Specify which part(s) of the system are affected by the stimulus (e.g., service, database, communication channel, processing pipeline). Your architecture must explicitly show how these artifacts are protected, adapted, or controlled.
- **Environment:** Describe the operational conditions under which the stimulus occurs (e.g., normal operation, degraded mode, peak load, partial failure, maintenance). Architectural decisions must be justified with respect to this environment, not an idealized one.
- **Responses:** Address **all** required system responses defined in the QAS. This typically includes both functional and quality-related behavior, such as detection, isolation, recovery, degradation, notification, enforcement, or prevention. Omitting a response is considered an incomplete solution.
- **Response measures:** For each response, explain how the architecture ensures that the corresponding quantitative or qualitative constraints are met (e.g., time bounds, availability percentages, throughput limits, retention periods, detection latency, recovery time). It must be clear which architectural mechanisms are responsible for meeting each measure.

Partial coverage (e.g., addressing only the normal execution path, or only one of several required responses) is insufficient. All responses and response measures must be explicitly traceable to architectural elements and illustrated where appropriate using sequence diagrams.

1.2 Architectural Trade-offs and Rationale

A central part of this assignment is the **explicit trade-off analysis with the initial architecture**. You are not only expected to explain *what* you changed or added, but *why* these changes were made and *what they cost* in terms of other architectural qualities.

Required analysis Your report and review must include a structured discussion that:

- **Identifies impacted architectural decisions from the initial architecture:** Explicitly reference the architectural decisions and rationale documented

for the initial architecture. Indicate which assumptions, constraints, or tactics are affected by your extension (e.g., centralized processing, synchronous communication, shared databases, trust assumptions).

- **Explains the relationship to existing decisions:** For each impacted decision, explain whether your extension:
 - reinforces or strengthens the original decision (e.g., same tactic, extended scope),
 - weakens or constrains it (e.g., reduced flexibility, tighter coupling, stricter timing),
 - or requires the decision to be revised or partially abandoned.

This must be argued explicitly, not implicitly assumed.

- **Justifies acceptability of the trade-offs:** Clearly argue why the introduced trade-offs are acceptable in the PMS context, given the QAS priorities, stakeholder concerns, and operational environment. If trade-offs introduce risks or limitations, these must be acknowledged explicitly.

Your rationale should be written in the same spirit and level of depth as the architectural rationale sections of the provided initial architecture. Arguments must be grounded in architectural principles and quality attribute reasoning, and must be traceable to concrete changes in the architecture diagrams and sequence diagrams.

1.3 Required Architectural Views

Your extension must be reflected consistently in the views described during the lectures and further expanded in the VPP exercise session. The purpose of these views is twofold: (i) to make the impact of your decisions explicit in the architecture, and (ii) ensure correctness, completeness, and consistency across views.

- **System context diagram:** Shows the PMS as a single system boundary and its *external* actors/systems (e.g., devices, clinicians, eHealth platform, third-party services) and the high-level interactions across the boundary. Use this view to make clear *what is outside your control* and where stimuli originate (many quality scenarios start outside the boundary).
- **Client–server view:** Shows the main runtime components/services of the PMS, their provided/required interfaces, and the communication relationships between them (who calls whom). Use this view to show how your extension changes the service interactions, responsibilities, and interface contracts (e.g., new service, new API, new data flow, added mediator, added gateway).
- **Process view (using sequence diagrams):** Uses quality-driven sequence diagrams to describe the *runtime behavior* of the system for the chosen QAS, including both the normal flow and the quality-related responses. Use this view to make explicit how the system detects and reacts to quality-relevant stimuli (e.g., failures, overload, violations), and how response measures are enforced (timeouts, retries, prioritization, isolation, recovery, auditing, etc.).

- **Decomposition view:** Shows the internal structure of a component by decomposing it into modules/subcomponents and their responsibilities and dependencies. Use this view to explain *how* a component achieves the QAS responses (e.g., adding a scheduler module, a cache, a policy module, an auditing module, a circuit breaker, a crypto module), and how responsibilities are separated.
- **Deployment view:** Shows the mapping of software elements to hardware/virtual nodes and the relevant communication links. Use this view to argue about quality attributes that depend on deployment choices (e.g., availability through replication, performance through locality, security/privacy through network zones and trust boundaries, failure isolation, scaling).

Architectural decisions that are not visible in the architecture diagrams (i.e., not reflected by changes in components, interfaces, responsibilities, or deployment) are insufficiently documented. Consistency is required: changes introduced in one view must be reflected in the other views wherever applicable, and must not contradict the sequence diagrams.

Sequence Diagrams (Quality-Driven) For each major response of the selected QAS, you must provide at least one **sequence diagram that illustrates the quality attribute response**.

- Sequence diagrams must go beyond normal functional flow.
- They must explicitly show:
 - detection of exceptional situations (e.g. overload, failure, violation),
 - the system’s response,
 - mechanisms enforcing response measures (timeouts, deadlines, retries, isolation, etc.).

2 Instructions

The goal of this part of the assignment is to **extend the initial software architecture for pms** so that it fully supports **two selected quality attribute scenarios (QAS)**. The focus is on **quality-driven architectural design**, not merely on adding functionality.

While the extension is driven by a quality requirement, you will necessarily have to complete or adapt *functional aspects* of the system in order to make the quality responses meaningful. These functional elements must be treated as supporting concerns and justified in terms of the chosen QASs.

You are explicitly expected to reason about the **architectural consequences of your decisions**, including how they interact with and impact the architectural decisions made in the initial architecture.

Design approach In addressing the selected QAS, you may choose between:

- a more systematic or structured approach (e.g. Attribute-Driven Design), or
- a more exploratory or ad hoc approach.

Regardless of the adopted approach, you are expected to plan your work carefully. Unstructured trial-and-error design and repeated backtracking are strong indicators of insufficient architectural reasoning.

You should explicitly identify early on which parts of the initial architecture are likely to be affected by your extension and which architectural decisions are likely to interact or conflict.

Report structure You are asked to compile a report that is structured as follows:

1. **Introduction** (optional)

A short introduction that positions the selected QAS and briefly outlines the scope of your extension.

2. **Architectural decisions for the selected QAS**

This section forms the core of the report and must be fully dedicated to the chosen QAS. It should be organized as a single, coherent subsection containing the following elements:

- (a) A brief overview of the *principal architectural decisions* introduced by your extension, highlighting the applied architectural tactics and patterns (if any).
- (b) A detailed discussion of the *architectural rationale*. Explain how and why the selected decisions support the QAS, explicitly referring to architectural elements (e.g. components, interfaces, nodes) and views (e.g. system context, client–server, decomposition, deployment). Use the **nameref** command to refer to diagrams and catalog elements where appropriate.

This discussion must be a *self-contained and complete explanation* of how the QAS is addressed, including all responses and response measures.

- (c) A short discussion of *alternative architectural decisions or approaches* that were considered. For each alternative, explain why it was re-

jected in favor of the final solution (e.g. insufficient QAS coverage, unacceptable trade-offs, increased complexity).

❗ Note: The primary goal of this section is to *explain architectural decisions and their rationale* (e.g. “we chose tactic X rather than Y to satisfy Av2 because ...”), *not* to describe or paraphrase diagrams. Diagrams serve as evidence of decisions, not as a substitute for rationale.

3. Architectural trade-offs and sensitivity points

Discuss the impact of your extension on the initial architecture. Identify key trade-offs and architectural sensitivity points, and explain how they influenced your design decisions. This section should clearly connect to the architectural rationale presented earlier.

4. Final architecture

Include the final architecture descriptions (diagrams and element catalog), generated from the final Visual Paradigm project using the SAPPlugin.

Formatting rules You must use the provided L^AT_EX template (automatically exported by the SAPPlugin). Diagrams and the element catalog must be generated using the Visual Paradigm SAPPlugin. Use the `\nameref` command to reference architectural elements and diagrams consistently.

Delivery The deadline for handing in Part 2 is **19/05**. You must submit:
(1) the architecture report (PDF), including diagrams and element catalog, and
(2) the Visual Paradigm Project (VPP) source file.

Good luck!
The Software Architecture team.

3 List of Quality Attribute Scenarios

3.1 Sec1: Patient identification (M)

The PMS needs to reliably identify the patient in several information flows, to correctly associate the different events (incoming sensor data, emergencies) to the corresponding patient. When receiving new sensor data readings, the system should correctly identify the wearable unit (at the basis of device identifier or the identifiers of individual sensors) and its association to the patient. In addition, the patient gateway should correctly present the patient's identity to the back-end services such that malicious actors cannot feed the system with false sensor data. Similarly, the PMS should be able to correctly identify the relevant patient when sending out notifications to other systems.

- **Source:**
 - Unregistered user or patient gateway
 - Registered patient gateway
- **Stimulus:**
 - A malicious actor presents crafted/falsified sensor data readings (cf. *UC4*: send sensor data) to the patient gateway
 - A malicious actor presents crafted/falsified sensor data readings (cf. *UC4*: send sensor data) to the PMS backend services.
 - A malicious actor falsely sends an emergency notification (cf. *UC3*: send emergency notification) via a patient gateway device that is unregistered or tampered with.
 - New sensor data readings (cf. *UC4*: send sensor data) or an emergency notification (cf. *UC3*: send emergency notification) for a patient is received via a patient gateway registered to another patient.
- **Artifact:** The subsystems that processes incoming requests.
- **Environment:** Normal operation
- **Responses:**
 1. The patient registration procedure ensures that the PMS backend has the necessary information to identify the patient gateway and the different sensors associated with the newly-registered patient's account: the device identifiers of wearable units, sensors, etc. are correctly linked to the patient's identity.
 2. Access to the PMS app is subject to authentication, i.e. a patient has to provide a username and password before further app functionality can be used.
 3. Interaction between the patient gateways and the backend system is subject to authentication, i.e. each patient gateway is authenticated before any further operations such as providing new sensor data readings or retrieving patient information are allowed.
 4. Patient information is subject to access control, i.e. patients can only consult their own information, only the treating physician can access patient data.
 5. The confidentiality and integrity for all information transmitted between the a patient's app and the PMS backend is actively preserved.
 6. Each attempt to interact with the backend system is logged, where each log entry contains
 - the time and date of the request,
 - identification of the requesting patient gateway and/or patient (before actual authentication, these are the IP addresses),
 - identification of the requested operation(s), and
 - whether the request was granted.
- **Response measures:**
 1. It is computationally infeasible to guess correct authentication information. For example, by relying on proper cryptographic keys and sufficiently long and random identifiers.
 2. It is computationally infeasible to bypass the authentication mechanism in the PMS app.
 - User passwords are enforced to be minimally 8 characters and maximally 64 characters long, and should *not* be commonly used passwords.¹
 - The identity of the patient is verified during the registration process.

¹Candidate passwords should be verified using libraries such as `zxcvbn` or `haveibeenpwned`.

3. It is computationally infeasible to spoof a registered app, e.g., by relying on proper public-key cryptography or secure API tokens.
4. It is computationally infeasible to bypass the access control mechanism. Each app only receives information that concerns the patient that is associated with the app in question.
5. It is computationally infeasible to breach the confidentiality of transmitted data or modify transmitted information without such modifications being detected.
6. Each request is logged before it is processed by the backend. Logs are kept for at least 2 years.

3.2 M1: Include third party lifestyle tracker data in clinical model computations (M)

Lifestyle trackers or ‘fitness monitoring devices’ (e.g. wristband sensors) are becoming increasingly popular, especially among those users that deliberately want to adopt an active and healthy lifestyle. In its initial development, the PMS directly collects the necessary data through the wearable unit dedicated for the concrete medical purpose of CVD monitoring. However, the data collected by these lifestyle tracker devices represent a wealth of complementary information that can be used for confirmation or contextualization (exercise regime, dietary habits, etc.). In the future, the PMS would like to be able to incorporate such additional data sources. Therefore, it should be easy to extend the PMS with support to interact with the backend services of different relevant lifestyle tracking apps, e.g., the data sharing services offered by Strava or Google Fit, to retrieve additional data related to a patient. This extension will only apply to patients that are also users of such lifestyle tracking devices and are willing to share such information, and such data intake from external sources can only be accomplished with explicit permission and configuration by the patient.

- **Source:** The research department
- **Stimulus:** Wishes to improve the accuracy of the clinical model computation and the resulting predictions by incorporating additional types of data.
- **Artifact:** This modification affects the clinical model computation, the patient dashboard, and communication subsystem(s).
- **Environment:** At design time or at runtime
- **Responses:**
 1. The patient app should be easily extensible with functionality to allow patients to give permission to the PMS to systematically retrieve their data from other systems, e.g., by allowing patients to set an appropriate authentication token.
 2. The PMS should be easily extensible with dedicated communication subsystem(s) to interact with different third-party backends, including support to regularly synchronize the relevant data.
 3. This modification should have no impact on the clinical model computation subsystem beyond potentially using a new model that accepts the new type(s) of data as additional input.
 4. This modification should not affect the existing interface(s) through which data is received, i.e. raw sensor data and information from the HIS.
 5. Storing the new types of data and the new data sets should not require changes to existing storage subsystems, e.g., the storage of (raw) sensor data readings or patient profiles.
 6. Additional attention must be paid to ensure that lack of or limited performance or availability of the third-party services will not affect the performance or availability of the risk estimations performed by our system.
- **Response measures:**
 1. The extension of the app should not take longer than 1 person month to develop, test, and deploy.
 2. Extending PMS with additional communication subsystem(s) to interact with third parties does not take longer than 3 person months to develop, test, and deploy.
 3. Once deployed, including another, additional third party service should not take longer than 2 person weeks to develop, test, and deploy.

3.3 P1: Data exchange with physicians (M)

The PMS exchanges data with several physicians, e.g., a cardiologist performs an on-demand consultation and the physicians registered for a patient need to be notified when their patient's risk level changes. Such exchanges of information should not hinder any medical services provided to the patient and should be conducted in a timely fashion.

- **Source:** Physician

- **Stimulus:**

- A physician consults a patient's status (cf. *UC6*: consult patient status), e.g., during a consultation with the patient.
- A cardiologist requests a patient's current data (cf. *UC9*: perform on-demand consultation), e.g., during a consultation with the patient.
- A cardiologist (re)configures the risk assessment for a patient (cf. *UC7*: configure patient risk assessment).
- A cardiologist updates the risk level of a patient (cf. *UC8*: update risk level), for example, after reviewing a received notification.
- The PMS looks up the appropriate recipients and sends a notification about a patient's status (cf. *UC5*: notify registered parties).

- **Artifact:** The subsystem(s) responsible to handle requests from physicians and send out notifications.

- **Environment:** Normal mode

- **Responses:**

1. The system is able to process all requests for patient information in a timely fashion.
2. Any risk assessment triggered by an on-demand consultation or risk assessment configuration is initiated within a short time frame of the request. In case of remote on-demand consultations, the results of the risk assessment are provided to the physician in a timely fashion.
3. The system is able to process all changes to a patient's risk level in a timely fashion.
4. The system is able to send notifications concerning (changes to) a patient's status to all appropriate recipients in a timely fashion, as long as the number of notifications to send stays within a threshold.
5. When the number of notifications to send exceeds the threshold the system prioritizes more urgent notifications.
6. A large number of notifications to be sent or requests arriving to the system should have no impact on the (a) the processing of incoming raw sensor reading data; or (b) ongoing risk estimations.

- **Response measures:**

1. The system is able to process at minimum 25 requests for patient information per minute (cf. *UC6*: consult patient status)
 - Requests for high-risk patients are processed in 2 seconds.
 - Requests for medium-risk level patients are processed in 5 seconds.
 - Requests for low-risk patients are processed in 10 seconds.
2. The system is able to process at least 20 requests for an on-demand consultation (cf. *UC9*: perform on-demand consultation) and/or risk assessment configurations per minute (cf. *UC7*: configure patient risk assessment).
 - The risk assessments triggered in an on-demand consultation are prioritized over regular risk estimations triggered by scheduled arrival of new sensor data.
 - Each risk assessment (cf. *UC12*: estimate risk level) triggered in an on-demand consultation is initiated within 3 minutes of receiving the request from the physician, independent of the patient's risk level.
 - In case of an on-demand consultation, the results of the risk assessment are provided to the physician within 1 minute after its completion.
3. The system is able to process 20 risk-level updates per minute (cf. *UC8*: update risk level).
 - Updates to a patient's EHR are forwarded within 1 minute.
4. The system should be able to process and send at least 100 notifications per minute (cf. *UC5*: notify registered parties).
 - A red notification should be sent to all recipients within 10 seconds.

- A yellow notification should be sent to all recipients within 30 seconds.
 - A green notification should be sent to all recipients within 1 minute.
5. When the system must send more than 100 notifications (cf. *UC5*: notify registered parties) in a minute, it prioritizes red notifications over yellow and green ones, and yellow notifications over green ones. No red or yellow notifications may be lost.

3.4 Av1: Internal PMS database failure (H)

The internal subsystem of the PMS that is responsible for storing the (raw) sensor data readings fails or crashes. Significant and/or frequent downtime of this data store is problematic, as it will impede the execution of risk estimation processes that rely upon historical sensor data and limit our capacity to persist new and incoming sensor data.

- **Source:** Internal
- **Stimulus:** The internal subsystem responsible for storing the sensor data readings fails or crashes.
- **Artifact:** Internal subsystem
- **Environment:** Normal execution
- **Responses:**
 1. This does not affect the availability of other types of persistent data, such as (a) the registered parties that should receive notifications, (b) the pairing of sensors, patient gateway and patient (and their enrollment), (c) the authentication credential data, (d) the risk estimation results, notifications, etc.
 2. A crash does not lead to any loss of already stored (raw) sensor data readings, to data inconsistency or lack of integrity.
 3. Prevention:
 - The storage subsystem should have a minimal guaranteed up-time.
 4. Detection:
 - In case a crash occurs, a PMS system administrator is notified of the problem.
 - The PMS is able to detect this problem and goes into degraded mode.
 - * Sensor data readings that arrive during downtime are temporarily stored elsewhere and are added to the storage subsystem when it returns operational.
 - * Requests for sensor data for clinical model computations fail gracefully, such that the computations can be rescheduled, or users can appropriately be informed.
 5. Resolution:
 - A PMS system administrator addresses the problem, e.g., by replacing failed hardware or by restarting software, and, if necessary, reverts the subsystem to a previous consistent state, using a backup.
 - The PMS automatically brings the repaired storage subsystem up to date by adding the (raw) sensor data readings that arrived while it was unavailable.
- **Response measures:**
 1. Prevention:
 - For high-risk patients, the raw sensor database should be available 99% of the time (measured per month).
 - For medium-risk patients, the raw sensor database should be available at least 95% of the time (measured per month).
 - For low-risk patients, the raw sensor database should be available at least 90% of the time (measured per month).
 2. Detection:
 - A PMS administrator is notified within 5 minutes of the detection of the failure.
 - Detection of failed hardware or crashed software happens within 1 minute of the failure or crash.
 - In case of high or medium patient risk status,
 - * there is a seamless transition to the temporary backup storage (no disruption);
 - * no data updates are omitted, regardless of the size or frequency of the updates (patient data readings typically represent detailed and thus high-volume data)
 - In case of a low risk level, at most two of the regular updates may be omitted.
 3. Resolution:
 - It should be possible to restart the storage subsystem within a minute, and
 - roll-back to a previous or backup state happens within 5 minutes.

3.5 Av2: Communication between patient gateways and PMS backend (H)

The PMS depends on the interoperation between the patient gateway and the backend services. Key to this interoperation is the communication channel between a patient gateway and the PMS backend, used to relay raw sensor data and emergency notifications. Vice versa, functionality such as the support for on-demand consultation relies upon a reliable communication channel between the PMS backend services and the patient gateway.

Key functionality of PMS can be compromised if (1) a part of the intermediate telecommunication infrastructure fails; (2) a patient gateway of one or more patients fails or crashes; or (3) internal communication subsystem(s) of PMS involved in the aforementioned interactions fail(s) or crash(es). For example, emergency notifications from a patient's gateway can no longer be received and sensor data readings can not be sent from the patient gateway(s) to the PMS for further analysis.

- **Source:** external or internal

- **Stimulus:**

- the (external) communication channel between a patient gateway and the PMS becomes unavailable, e.g., due to failure or (scheduled) maintenance;
- a patient gateway becomes unavailable, e.g., its battery may have run empty or it has no network connection; or
- an internal communication subsystem of the PMS fails or crashes.

- **Artifact:** external communication channel(s), external device(s), internal subsystem(s)

- **Environment:** Normal executions

- **Responses:**

1. Prevention:

- PMS has negotiated a Service-Level Agreement (SLA) with the intermediate telecommunication operator that stipulates
 - * an availability of at least 99.5% (measured per year) of the communication channel is guaranteed;
 - * at least 80% mobile network coverage in the broad region in which the PMS will operate; and
 - * an average bandwidth of at least 64Kb/s.
- The app on the patient's gateway warns the patient in time when the battery of their patient gateway unit is running low.
- The app on the patient's gateway warns the patient when they have no or limited network connectivity.

2. Detection:

- The PMS back-end services should be able to autonomously detect any failures based on the lack of sensor data updates from one or more patient gateways.
- The PMS back-end services should be able to autonomously detect any failures of the relevant internal communication subsystems.
- The patient gateway app should be able autonomously detect any communication failures and goes into degraded mode, which involves temporarily storing sensor data and notifications for later synchronization, systematic retrying to complete the communications, and use of possible back-up communication channels.
- The PMS keeps track of how long there was a lack of communication.

3. Resolution:

- The PMS proactively notifies the relevant stakeholders of the detected problem.
 - * In case of a failing communication subsystem, the system administrator must redeploy the failing component(s), or revert it to a previously working state.
 - * In case of a failing communication channel, the system administrator must contact the telecommunication operator to resolve this.
 - * In case of a failing patient gateway, the patient should be contacted to resolve the issues.
- In degraded mode, the patient gateway application
 - * uses a back-up communication channel for emergency notifications, e.g., SMS; and
 - * keeps retrying to deliver sensor data readings in a proper fashion, e.g., using exponential back-off; and

- * the patient gateway is able to store the sensor data readings (and unsent notifications) for the past 24 hours.

- **Response measures:**

1. Prevention:
 - The app warns the patient when the battery of their patient gateway has dropped to 15% or less charge.
2. Detection:
 - Detection time depends on the transmission rate configured on the patient gateway (and indirectly on the risk level), but does not exceed this with more than 5 minutes. So, if an expected update does not arrive at expected time T , detection must have taken place before $T + 5$ minutes.
 - The PMS detects any failures of internal communication subsystems within 5 seconds after the failure.
3. Resolution:
 - Once detected, in 90% of the cases, the notifications sent to the system administrator arrive within 2 minutes if the patient has a high risk level, within 5 minutes if the patient has a medium risk level, and within 10 minutes in case of a low risk level.
 - Redeployment or roll-back of the communication subsystem does not take longer than 5 minutes.
 - The SLA with the telecommunication operator stipulates availability of technical support within 10 minutes and resolution within the hour.

3.6 Priv1: Right of access requests by patients (M)

Legislation such as the GDPR provides users with a “right of access” to their personal data. In summary this means that users have a right to obtain a copy of their personal data that is processed by the system along with information about the processing of this data, for example the purpose for processing and an overview of recipients. In the PMS, responding to such requests requires collecting data from, potentially different, subsystems (e.g., patient profile information and sensor data readings) as well as potentially interacting with external systems (e.g., with the HIS to obtain the data used from the patient’s health record). While it is paramount for PMS to comply to appropriate legislation (such as the GDPR), care should be taken to limit the impact on core functionality such as risk estimations.

- **Source:** User
- **Stimulus:** A user requests access to their personal data being collected and processed by the PMS.
- **Artifact:** Internal subsystem
- **Environment:** Normal mode
- **Responses:**
 1. The system can process multiple “right of access” requests simultaneously.
 2. The system can process all received requests within a reasonable time frame.
 3. The replied data archive should cover all relevant personal data that the system has processed up to, at least, the time the request was received (accuracy and completeness). This includes at least:
 - the patient’s profile information, including personal data kept on the patient Gateway;
 - all (past) risk estimations;
 - all sensor data readings processed by the system;
 - data used from the patient’s electronic health record; and
 - patient-specific configuration for the risk estimations, for example a clinical model tailored to the patient in question.
 4. For all personal data in the constructed archive additional information must be included on
 - the purpose(s) of processing that data;
 - any additional recipients to which the data has been or will be disclosed;
 - an indication of how long the system will store that data;
 - the source of the personal data, in case it was not collected from the patient themselves.
 5. The system ensures that the constructed data archive remains available for download by the patient for a sufficiently long time.
 6. Collecting the personal data and additional data to construct the archive necessary to respond to access requests has no impact on
 - processing incoming sensor data (*UC4*: send sensor data);
 - ongoing risk estimations (*UC12*: estimate risk level);
 - processing emergency notifications (*UC3*: send emergency notification); or
 - on-demand patient consultations (*UC9*: perform on-demand consultation).
 7. The system verifies the user’s identity prior to processing a personal data request and when a user requests to download a personal data archive.
- **Response measures:**
 1. The system can process at least 3 “right of access” requests simultaneously.
 2. For each request, the system should be able to
 - reply whether or not personal data is, or has been, processed concerning the requesting user within 1 hour after receiving the request; and
 - make an archive available that contains a copy of all relevant personal data within 48 hours after receiving the request.
 3. The generated archive remains available for download by the requesting user for at least 1 week after it was made available.